



1. TỔNG QUAN MẠNG MÁY TÍNH & LẬP TRÌNH MẠNG

GV. Trần Thị Minh Khoa

1

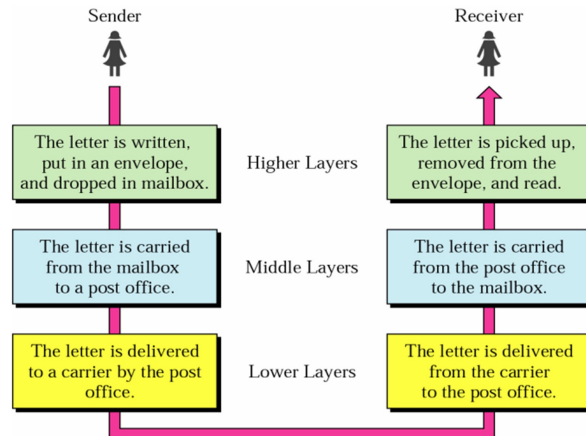
1

Tài liệu tham khảo:

1. Jan Graba, An Introduction to Network Programming with Java, Springer, 2007
2. Elliotte Rusty Harold, java network programming, 3rd Edition, O'Reilly, 2004

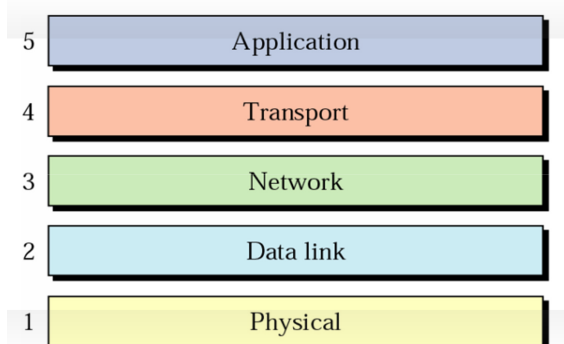
2

Trao đổi thông tin trên thực tế



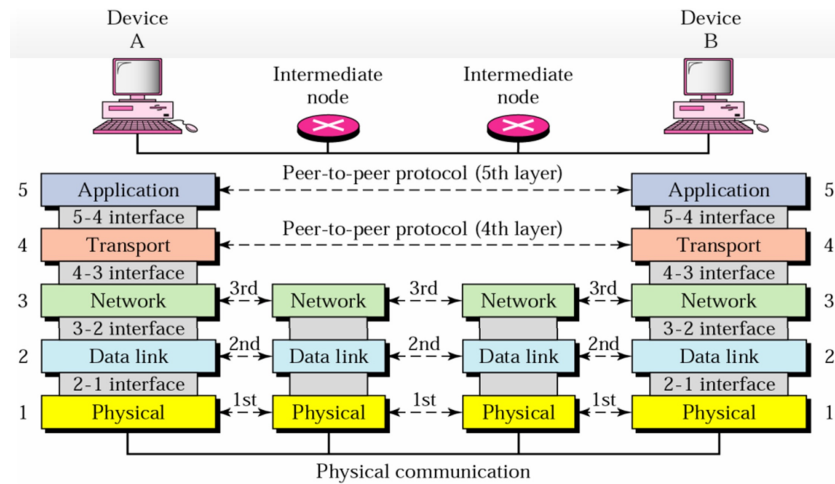
3

Các lớp giao thức Internet



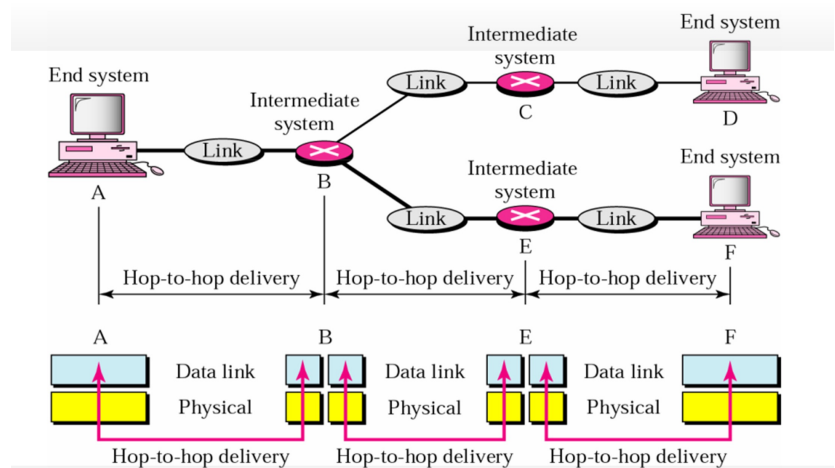
4

Giao tiếp trên hệ thống truyền thông



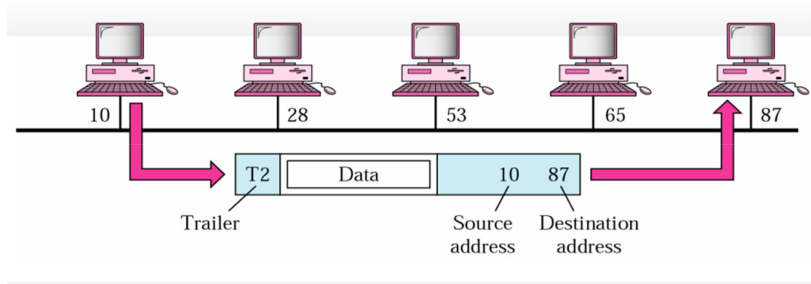
5

Kết nối qua từng chặng



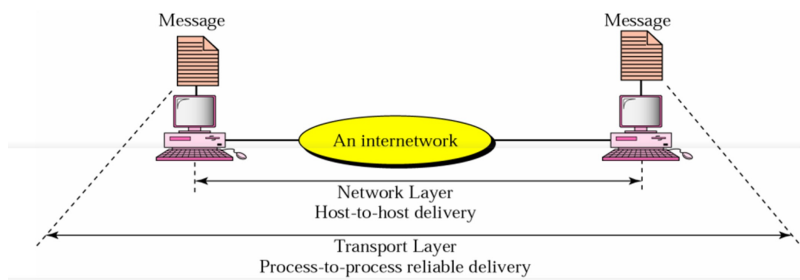
6

Giao tiếp trên mỗi chặng



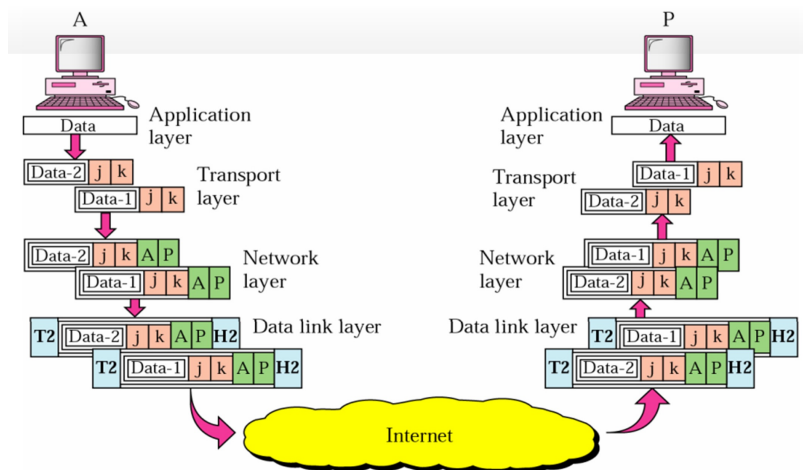
7

Chuyển phát tin cậy một thông điệp



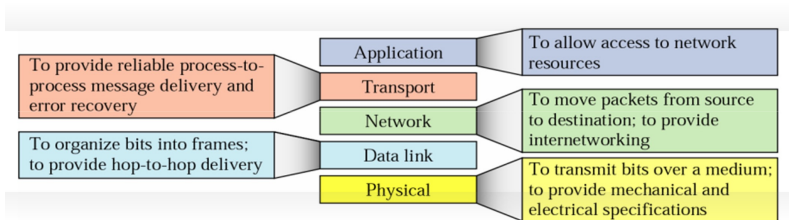
8

Ví dụ



9

Tóm tắt nhiệm vụ của từng lớp giao thức



10

Hai chế độ hoạt động của giao thức

- **Connection-oriented (Connection-based)**
 - Giao tiếp ngang hàng có duy trì một đường đi cố định
- **Connectionless (Datagram)**
 - Giao tiếp ngang hàng không có đường nối cố định
 - Mỗi gói tin có thể đi trên các con đường khác nhau đến đích

11

Tổng quan lập trình mạng (1/2)

- Mạng máy tính có hai phần
 - Vật lý (phần cứng)
 - Luận lý (phần mềm)
- Giao tiếp vật lý
 - Chuẩn giao tiếp vật lý
 - Chip logic có thể lập trình
 - Chương trình điều khiển (driver hay có sẵn trong BIOS, trong hệ điều hành)
 - Lập trình: các driver, các **API**

12

Tổng quan lập trình mạng (2/2)

- Giao tiếp luận lý
 - Giao thức mức thấp: Đặc tả giao thức trong các RFC → lập trình giao thức
 - Giao thức mức cao: Đặc tả giao thức trong các RFC → lập trình giao thức.
 - **Ứng dụng mạng**: Các chương trình tiện ích của người dùng trao đổi thông tin hay sử dụng dịch vụ qua mạng → lập trình dựa vào API

13

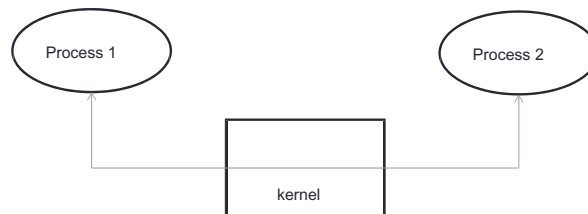
Giao tiếp liên tiến trình

- Process
- Hệ thống đa nhiệm
- Cơ chế cho phép giao tiếp giữa các process trong hệ thống đa nhiệm: IPC (Inter-Process Communication)

14

Hai loại giao tiếp IPC

- Các process trên cùng một máy

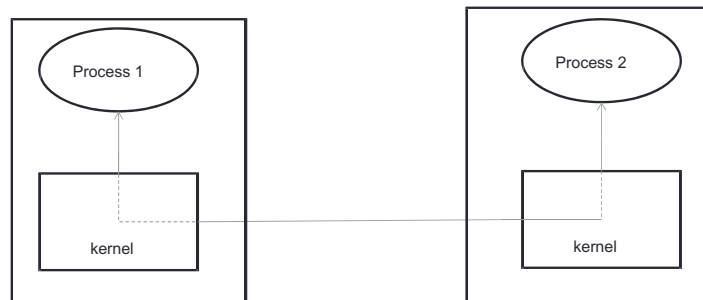


15

Hai loại giao tiếp IPC

- Hai tiến trình trên hai máy khác nhau

*Các kernel thoả thuận với nhau các quy tắc trao đổi thông tin.
Các quy tắc là một phần của giao thức*



16

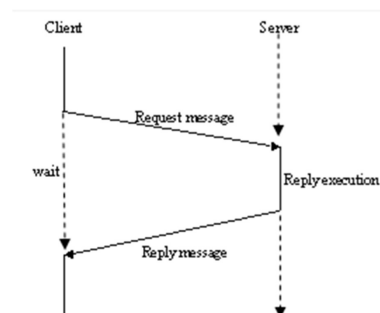
Ứng dụng mạng

- Chương trình tạo ra các tiện ích (dịch vụ) phục vụ người dùng
 - Tiện ích về lưu trữ và truy xuất tập tin (ftp)
 - Tiện ích đầu cuối từ xa (telnet)
 - Tiện ích in ấn
 - Tiện ích duyệt web
- Ứng dụng mạng được xây dựng dựa vào mô hình ứng dụng : **peer-to-peer, client-server, lai ghép.**

17

Mô hình client-server

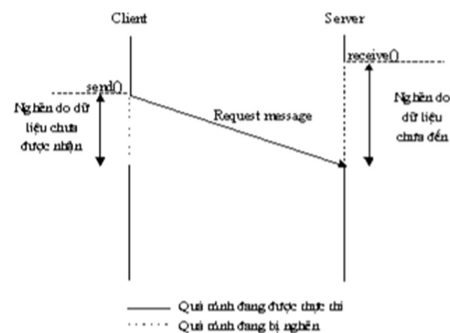
- Ứng dụng cấu thành từ hai phần:
 - Quá trình chuyên cung ứng sự phục vụ là server
 - Quá trình chuyên yêu cầu phục vụ từ server gọi là client



18

Hai chế độ giao tiếp

- **Chế độ blocking** (khóa/nghe): khi client hay server phát ra lệnh send gửi thông điệp, quá trình client hay server tương ứng sẽ bị treo cho đến khi phía nhận phát lệnh receive tiếp nhận thông điệp. Tương tự khi gọi lệnh receive mà chưa có send cũng sẽ vào trạng thái treo cho đến khi có lệnh send từ xa gửi thông điệp đến.



19

- **Chế độ nonblocking**: khi phát ra lệnh truyền thông, quá trình client hay server vẫn tiếp tục được thực thi mà không bị treo

Hai chế độ giao tiếp (tt)



20

Kiến trúc chương trình

- Các chức năng trong một chương trình ứng dụng gồm :
 - Giao diện nhập liệu (user interface service)
 - Tính toán, xử lý thông tin theo qui định (business rule service)
 - Truy vấn và lưu trữ thông tin (data storage service)
- Ở mức vật lý các chức năng này được lập trình trong một hay nhiều tập tin tùy theo kiểu kiến trúc :
 - Kiến trúc một tầng
 - Kiến trúc hai tầng
 - Kiến trúc ba tầng

21

Kiến trúc một tầng (single-tier architecture)

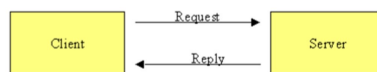
- Cả ba chức năng của phần mềm ứng dụng đều được lập trình trong cùng một tập tin



22

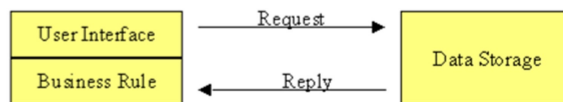
Kiến trúc hai tầng (two-tier architecture)

- Được gọi là kiến trúc client server
- Gồm 2 chương trình thực thi
- Trên hai máy tính khác nhau hay trên cùng một máy tính
- Mỗi ứng dụng phải có giao thức (protocol) được định nghĩa để giao tiếp trao đổi thông tin

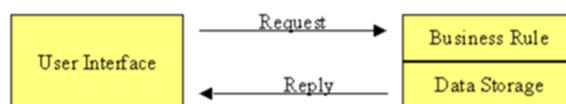


- Thông thường client đảm nhận user interface
- Server đảm nhận data storage
- Chức năng business rule có thể
 - Lập trình tạo ra trên client □ Fat client
 - Lập trình tạo ra trên server □ Fat server

23



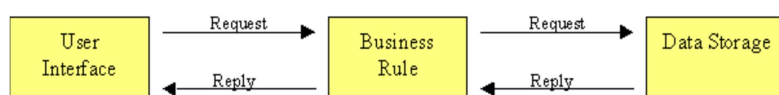
Fat Client



Fat Server

24

Kiến trúc 3 tầng



25

Nhắc lại, NGÔN NGỮ JAVA

26

26

Đặc điểm

- Ngôn ngữ cấp cao
- Thuần hướng đối tượng
- Có thể thực thi trên các hệ điều hành khác nhau
- Đa luồng
- Phân tán
- Ngôn ngữ động
- Đơn giản, dễ học, kiến trúc chương trình dễ hiểu

27

Máy ảo java (JMV)

- Đảm bảo cho chương trình chạy trên nhiều nền tảng khác nhau.
- ByteCode là ngôn ngữ máy của máy ảo giống các mã lệnh nhị phân của máy thật.
- Một chương trình được viết được lưu thành tập tin có phần mở rộng là .java, phải được biên dịch thành tập tin thực thi trên máy ảo java có phần mở rộng là .class.
- Tập tin thực thi chứa các chỉ thị ở dạng ByteCode mà máy ảo java hiểu được.
- Khi thực thi chương trình, máy ảo dịch các ByteCode thành mã nhị phân của máy tính thật
- => máy ảo là trình biên dịch

28

Hai kiểu ứng dụng

- Applet
 - Chương trình nhúng vào trang web, được tải từ webserver về máy client
- Application
 - Chương trình thực thi trực tiếp trên máy ảo java

29

Bộ công cụ phát triển ứng dụng JDK

- Được cung cấp miễn phí bởi JavaSoft (Sun), javasoft.com
- Mỗi loại hệ điều hành sẽ có bộ JDK tương ứng
- Các chương trình thực thi quan trọng của JDK
 - **javac**: biên dịch chương trình nguồn thành tập tin thực thi trên máy ảo
 - **java**: làm cho máy ảo biên dịch mã ByteCode thành mã thực thi trên máy thật
 - **appletviewer**: thông dịch, thực thi các chương trình applet
 - **jdb**: trình gỡ rối
 - **javadoc**: tự động tạo tài liệu chú thích chương trình nguồn
 - **rmic**: tạo Stub cho ứng dụng kiểu RMI
 - **rmiregistry**: thực hiện danh bạ trong RMI

30

Kiểu dữ liệu

- Số:

Tên kiểu	Kích thước
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes

- Ký tự char
 - 2byte
- Kiểu chuỗi String
 - Là lớp thuộc thư viện chuẩn của java, java.lang.String
- Kiểu logic boolean: 2 giá trị true và false
- Kiểu mảng
 - khai báo `int[] a;` hay `int a[];` `int maTran[][]`
 - Khởi tạo biến `a= new int[10];` `maTran=int[9][9]`
 - Sử dụng mảng `int i=a[10];` `int x=maTran[2][5]`

31

Các phép toán cơ bản

Tương tự như C++

- Toán số học: `+, -, *, /, %, =, ++, --, +=, -=, /=, %=`
- Toán logic: `==, !=, &&, ||, !, >, <, >=, <=`
- Toán trên bit: `&, |, ^, <<, >>, ~`
- Điều kiện: `?:`
- Chuyển đổi kiểu (ép kiểu): (kiểu mới)

32

Nguyên tắc đặt tên

- Tên phân biệt chữ hoa và chữ thường
- Dùng chữ cái, chữ số, ký tự _ và dấu \$
- Không bắt đầu bằng chữ số

Tên lớp: Viết hoa các ký tự đầu của

từ Ví dụ: InputStream, WhileDemo

Tên biến, tên hằng hay tên phương thức: từ đầu tiên viết thường, các từ tiếp theo viết hoa ký tự đầu

Ví dụ maLop, giaTriDau, getInputStream(), maLop,

33

Simple program

```
App.java
// Chương trình thuộc package nào (chỉ thuộc 1 package)
package package_name;

// Import class vào để sử dụng (có thể import nhiều class)
import java.util.Scanner; // Import một class
import java.util.*; // Import toàn bộ package

// Định nghĩa thêm các class khác
class Util {
    ...
}

// Class chính, là public class duy nhất trong file
public class App {
    ...
    // Class chính phải có method main() như sau
    public static void main(String[] args) {
        ...
    }
}
```

34

34

```

App.java
// Biến không khai báo ở đây
public class App {
    // Biến thực thể
    int instanceVar = 10;

    // Biến tĩnh
    static double staticVar; // Không cần init value

    // Đây là method
    public static void main() {
        // Biến cục bộ
        int localVar = "Hello", name = "John";
    }
}

```

35

35

Chương trình ứng dụng kiểu Application

- Ngôn ngữ thuần đối tượng
- Một chương trình ứng dụng chỉ có một lớp thực thi
- Lớp thực thi phải có tên trùng với tập tin chứa nó, phải khai báo public và chứa phương thức:

```

public static void main (String[] args) {
    .....
}

```

Là phương thức được thực thi đầu tiên

36

Cài đặt hệ thống

- Download JRE và JDK từ javasoft.com phù hợp với hệ điều hành
- Cài đặt, chọn nơi cài đặt ví dụ C:\jdk1.6.0_33
- Thiết lập biến môi trường
 - PATH = C:\jdk1.6.0_33\bin;
 - CLASSPATH = C:\jdk1.4\lib,;.

37

Lập trình

- HelloWorld
- Dùng JDK để biên dịch và thực thi

```
public class HelloWorld {  
    public static void  
    main(String args[]) {  
  
        System.out.print("Hello World!\n");  
    }  
}
```

38

Ví dụ in ra màn hình

Dùng

`System.out.print` (tham số 1+ tham số 2 + tham số 3 + ...)

```
public class InManHinh {
    public static void main(String args[]) {
        int i= 1234;
        String str="    la Lap trinh mang";
        char ch = 'A';
        System.out.print('\n'+ "Ma mon hoc:"
+ch+ i + str);
    }
}
```

39

Ví dụ nhập ký tự từ bàn phím

Dùng `System.in.read()`;

```
import java.io.*;
public class KeyRead {
    public static void main(String args[]) {
        try {
            int ch = System.in.read();
            System.out.print("Ky tu " + (char)ch + "
co ma ascii = "+ch);
        } catch(IOException ie) {
            System.out.print("Error " + ie) ;
        }
    }
}
```

40

Cấu trúc if

Cú pháp:

```
if (Condition) {
    // Các lệnh sẽ được thực hiện nếu giá
    trị của Condition là true
}
if (Condition) {
    // Các lệnh sẽ được thực hiện nếu giá trị của
    Condition là true
} else {
    // Các lệnh sẽ được thực hiện nếu giá trị của
    Condition là false
}
```

41

Ví dụ lệnh if

```
import java.io.*; public class IfDemo
{
    public static void main(String args[]) {
        System.out.print("Vui long nhap mot ky tu:"); try {
            int ch = System.in.read(); if (ch ==
                'A') {
                System.out.print("Ban rat may man !");
            }
            else {
                System.out.print("Ban khong gap may !");
            }
        } catch(IOException ie) {
            System.out.print("Error:"+ie);
        }
    }
}
```

42

Lệnh while

Cú pháp

```
while (condition) {
    // nếu condition có giá trị là true, thì các tác vụ ở
    // đây sẽ được lặp lại
}
```

Ví dụ

```
import java.io.*;
public class WhileDemo {
    public static void main(String
        args[]) { int num = '9';
        while (num > '0') {
            System.out.print((char)num + "
                "); num--;
        }
    }
}
```

43

Lệnh do-while

Cú pháp

```
do {
    // Lặp lại các tác vụ ở đây cho đến khi điều kiện
    // condition có giá trị là false
} while (condition)
```

Ví dụ

```
import java.io.*;
public class DoWhileDemo {
    public static void
        main(String args[]) { int
            num = '9';
            do {
                System.out.print((char)num
                    + " "); num--;
            } while (num > '0');
        }
}
```

44

Lệnh for

Cú pháp

```
for (operation1; condition; operation2){
    // Các tác vụ được lặp lại
}
```

Tương đương như cấu trúc sau: operation1;
while (condition) {
 // Các tác vụ được lặp lại
 operation2;
}

45

Ví dụ lệnh for

```
import java.io.*; public
class ForDemo {
    public static void main(String args[]) {
        {
            for(int num = '9'; num>'0'; num --)
                System.out.print((char)num + " ");
        }
    }
}
```

46

Bài tập

- Tìm một ứng dụng mạng bất kỳ và mô tả :
 - Các chức năng hỗ trợ của ứng dụng
 - Kiến trúc của ứng dụng
 - Giao thức hỗ trợ ứng dụng
 - Xác định các trạng thái của các máy và sơ đồ hoạt động chuyển đổi giữa các trạng thái (Chu trình hoạt động của mỗi máy liên quan trong ứng dụng mà qua đó xác định giải thuật của chương trình)

47

MỘT SỐ KỸ THUẬT THÔNG DỤNG

48

48

Nội dung

- **STREAM (dòng chảy)**
 - `java.io.InputStream`
 - `java.io.OutputStream`
- PIPE (đường ống lệnh)
 - `java.io.PipedInputStream`
 - `java.io.PipedOutputStream`
- **THREAD VÀ MULTI-THREAD**

49

Kỹ thuật giao tiếp dùng Stream

- Khái niệm Stream
- Input Stream và Output Stream
- Stream trong java

50

Khái niệm Stream

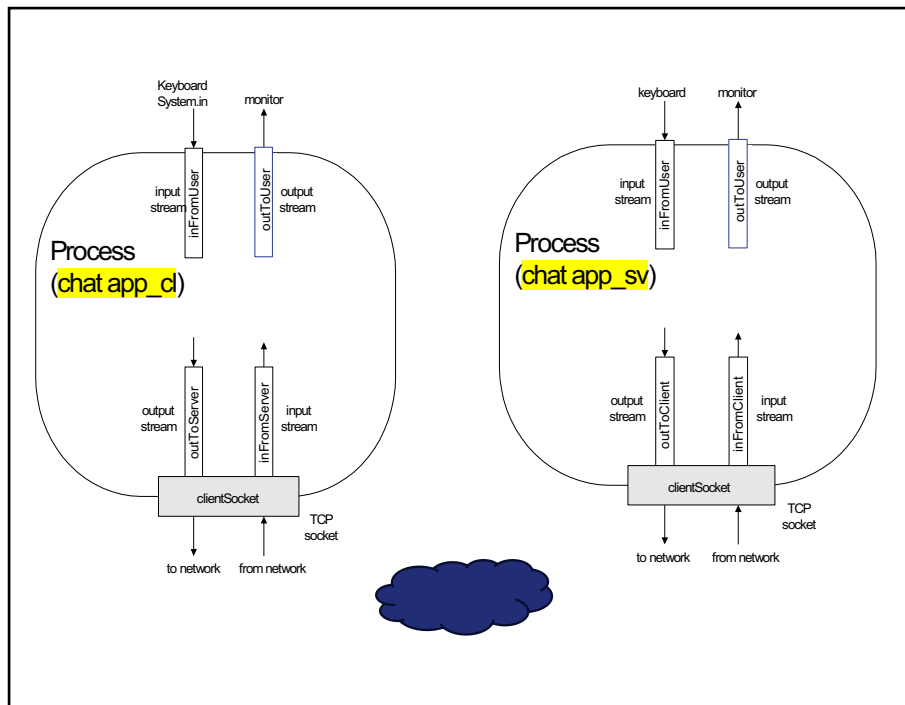
- Stream là dòng liên tục, có **thứ tự của các bytes dữ liệu** chảy giữa chương trình/ứng dụng và các thiết bị ngoại vi
- Dùng stream có thể kết nối nhiều thiết bị ngoại vi với chương trình

51

Input Stream và Output Stream

- Nếu dòng dữ liệu trong Stream có hướng chảy từ thiết bị ngoại vi vào chương trình thì ta nói đây là Stream nhập (Input Stream)
- Chảy theo chiều ngược lại là Stream xuất (Output Stream)

52



53

Cách giao tiếp dùng stream

- Để giao tiếp, trước tiên chương trình phải lấy được các stream nhập / xuất gắn với thiết bị.
- Sau đó,
 - chương trình có thể gửi dữ liệu ra ngoại vi bằng thao tác ghi vào Stream xuất của thiết bị.
 - có thể nhận dữ liệu từ ngoại vi bằng thao tác đọc Stream nhập của thiết bị .

54

Stream trong java

- Đối với Java, các thiết bị đều có Stream nhập/Stream xuất nối với nó.
- Java hỗ trợ hai các lớp stream cơ bản trong gói java.io là:
 - java.io.InputStream: Stream nhập → System.in
 - java.io.OutputStream: Stream xuất → System.out

55

Lớp java.io.InputStream

Là lớp nhận dữ liệu từ bên ngoài

- Có phương thức:
 - `int read() throws IOException`
 - Đọc 1 byte từ Stream
 - Return 0-255 : Mã ASCII của byte nhận được từ ngoại vi
 - 1 : Stream đã kết thúc, không còn dữ liệu.
- Đối với java `System.in` là một InputStream nối với bàn phím

56

Vd 1b: InputStreamReader

```

import java.io.IOException;
import java.io.InputStream;

public class InStream1 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        InputStream is = System.in;
        while(true){
            try {
                int ch = is.read();
                if(ch == -1 || ch == 'q'){
                    break;
                }
                System.out.println((char)ch);
            } catch (IOException ioe) {
                System.out.println(ioe);
            }
        }
    }
}

```

57

57

Lớp java.io.InputStream (tt)

int read(byte b[]) throws IOException

Đọc tất cả các byte hiện có trong Stream vào mảng b.

Return 0-255: Số lượng byte đọc được.

-1 : Stream đã kết thúc, không còn dữ liệu.

int read(byte b[], int offset, int len)

Đọc len byte từ Stream hiện tại, lưu vào trong mảng b bắt đầu từ vị trí offset

Return: số lượng byte đọc được.

-1 : Stream đã kết thúc.

int available(): Trả về số lượng byte hiện có trong Stream mà không làm nghẽn chương trình

58

Vd2b: BufferedReader

```

import static java.lang.Thread.*;

import java.io.IOException;
import java.io.InputStream;

public class InStream2b {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) throws InterruptedException, IOException {
        InputStream is = System.in;

        try {
            while (true) {
                // Kiểm tra xem có bao nhiêu byte dữ liệu có sẵn mà không bị chặn
                if (is.available() > 0) {
                    byte[] buffer = new byte[is.available()];
                    int bytesRead = is.read(buffer);
                    if (bytesRead == -1) {
                        break; // Kết thúc input stream
                    }
                    String str = new String(buffer, offset:0, bytesRead);
                    System.out.print(str); // In ra dữ liệu đọc được
                } else {
                    // Nếu không có dữ liệu sẵn có, in một ký tự '.' và đợi một chút
                    System.out.print(c:'.');
                    sleep(millis:100); // Ngủ một chút để giảm tải CPU
                }
            }
        } catch (IOException e) {
        }
    }
}

```

59

59

Lớp java.io.OutputStream

- Cho phép chương trình xuất dữ liệu ra thiết bị ngoài có các phương thức:

void write(int b) throws IOException

- Viết byte b vào Stream hiện tại,
- Return : void

void write (byte[] b) throws IOException

- Viết tất cả các phần tử của mảng b vào Stream hiện tại
- Return : void

void write (byte[] b, int offset, int len) throws IOException:

- Viết len phần tử trong mảng b vào Stream hiện tại, bắt đầu từ phần tử có chỉ số là offset của mảng.
- Return : void

System.out là một OutputStream nối với màn hình

60

Nhập chuỗi từ một InputStream

- Vấn đề: chỉ cung cấp phương thức đọc byte và mảng byte
- Lớp `java.io.InputStreamReader`: Là cầu nối để chuyển InputStream dạng **byte** sang InputStream dạng các **ký tự**
- Lớp `java.io.BufferedReader`: Hỗ trợ việc **đọc** văn bản từ một InputStream dạng **ký tự**
- Phương thức String `readLine()` throws IOException của `BufferedReader` cho phép đọc dòng văn bản kế tiếp trong InputStream

61

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class ReadLineDemo {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        InputStream is = System.in;
        InputStreamReader isr = new InputStreamReader(is);
        BufferedReader br = new BufferedReader(isr);
        while (true) {
            try {
                String line = br.readLine();
                if (line == null) break;
                System.out.println(line);
            } catch (IOException e) {
                System.out.println(e);
            }
        }
    }
}
```

62

62

Xuất chuỗi ra một OutputStream

- Vấn đề: tương tự InputStream
- Để có thể gửi được chuỗi ra một OutputStream ta phải sử dụng lớp `java.io.PrintWriter`
- Phải đổi OutputStream dạng byte sang dạng ký tự, và viết chuỗi vào OutputStream dạng ký tự
- Ví dụ in chuỗi ra màn hình

63

Ví dụ 4: in chuỗi ra màn hình

```
import java.io.*;

public class PrintString {
    public static void main(String args[]) {
        OutputStream os = System.out;
        PrintWriter pw = new PrintWriter(os);
        pw.write("This is a string \r\n");
        pw.println("This is a line");
        pw.write("Bye! Bye!");
        pw.flush();
    }
}
```

64